

Análisis y diseño con patrones

PEC0: Modelado con UML

Índice

Compromiso.....	2
Pregunta 1 (20%).....	3
Pregunta 1.1 (10 %).....	3
Pregunta 1.2 (10 %).....	4
Pregunta 2 (40 %).....	6
Pregunta 2.1. (40 %).....	6
Pregunta 3 (40 %).....	7
Pregunta 3.1. (40 %).....	7

Compromiso

Certifico que he realizado la PEC0 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Pregunta 1 (20%)

Pregunta 1.1 (10 %)

El diagrama de clases se compone de los siguientes elementos:

- Clase **Book** (libro)

Es la clase que contiene los metadatos de cada libro del catálogo: ISBN, título (title) y autor (author). Su rol es fundamental para evitar redundancia de datos en el sistema, ya que no es óptimo repetir el título y el autor mil veces si tenemos mil copias del mismo libro. Por ello, la clase BookCopy se compone de esta clase. Estratégicamente, esto puede permitir la integración con bases de datos externas y la normalización del inventario.

Si eliminamos un libro del catálogo, sus copias físicas dejan de tener sentido en el sistema. La multiplicidad 1..* indica que para que un libro exista en el catálogo, debe haber al menos una copia física registrada.

- Clase **BookCopy** (copia de un libro)

Representa el inventario físico de libros de la biblioteca. Tiene una relación de composición con la clase Book, lo que significa que BookCopy no puede funcionar de forma independiente a Book. Un libro puede tener una o varias copias, mientras que una copia solo puede ser de un libro.

Mientras que el ISBN identifica al libro del que se compone, el código de barras (barcode) identifica a la copia específica. Cuenta con una operación que comprueba si la copia está disponible para préstamo (isAvailable()).

- Clase **Loan** (préstamo)

Representa un evento temporal del sistema que relaciona a los socios con el inventario físico de libros (**BookCopy**). Contiene tanto la fecha de préstamo como la fecha de devolución en la que se tiene que devolver la copia prestada. Como hemos visto en la clase **Member**, un préstamo debe pertenecer a un socio. A su vez, un préstamo específico se relaciona con una única copia de un libro.

- Clase **Member** (socio)

Cada socio se registra con un nombre y se le asigna un identificador alfanumérico. Se asocia a la clase Loan (préstamo): un socio puede tener muchos préstamos o ninguno, pero cada préstamo debe pertenecer obligatoriamente a un socio. Esto implica que nunca habrá un libro prestado a alguien no identificado.

Pregunta 1.2 (10 %)

El diagrama modela el proceso en el que un bibliotecario gestiona el préstamo de la copia de un libro. Las *lifelines* que intervienen son las siguientes:

- **Actor (librarian/bibliotecario):** Actúa como el controlador principal de la interacción. Es el intermediario entre el usuario físico y el sistema de software. Su línea de vida coordina todo el flujo de mensajes.
- **Instancia member de la clase Member (socio):** Representa al usuario de la biblioteca que desea llevarse el libro. El sistema interactúa con este objeto para validar su información.
- **Instancia book de la clase BookCopy (copia de un libro):** Representa el ejemplar físico específico que se va a prestar. Es fundamental que sea una copia y no un libro de la clase Book, ya que la disponibilidad recae sobre el objeto físico.
- **Instancia loan de la clase Loan (préstamo):** Representa el registro de la transacción. Si nos fijamos en su línea de vida, apreciamos que no comienza en la parte superior, sino más abajo, lo que indica que se crea dinámicamente durante el proceso solo si se cumplen ciertas condiciones.

A partir de estas entidades, se deriva una secuencia de mensajes que sigue un orden lógico diseñado para validar las precondiciones antes de ejecutar cualquier transacción que altere el estado del sistema.

Distinguimos estos mensajes:

1. **Call message getId():** El bibliotecario consulta a la clase Member para obtener su identificador.
2. **Return message member:** El call message getId() devuelve la información del socio al bibliotecario tras obtener su identificador.

Justificación: Antes de prestar un libro, el sistema necesita saber con certeza quién lo está solicitando para asociar el préstamo a un usuario válido.

Tras obtener correctamente al socio, identificamos estos mensajes:

3. **Call message isAvailable():** El bibliotecario consulta a la clase BookCopy si el ejemplar específico está libre mediante el método isAvailable().
4. **Return message isAvailable:** El call message isAvailable() devuelve un booleano al bibliotecario que determina si el libro está prestado o no.

Justificación: Si no se hace esta comprobación, el sistema podría intentar prestar un libro que ya está prestado, extraviado o en reparación.

A partir de este mensaje, entramos en un recuadro etiquetado como alt (alternative), el cual representa una estructura de control de flujo tipo if/else. Divide el comportamiento del sistema en

dos caminos mutuamente excluyentes basados en el resultado del mensaje 4 (isAvailable). Distinguimos dos caminos:

- **Camino exitoso [isAvailable == true]**

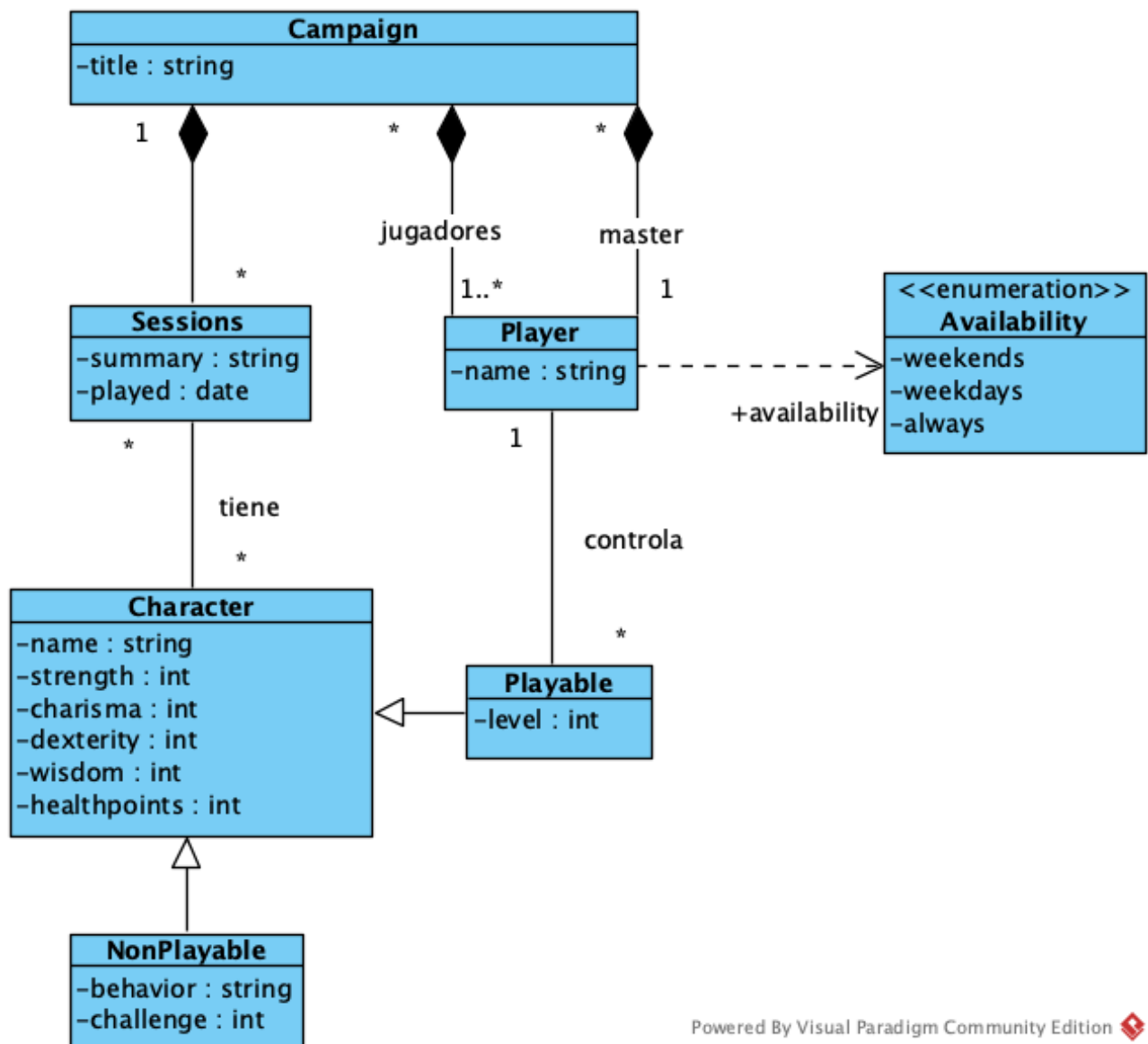
5. **Call message new Loan(member, book):** El sistema instancia un nuevo objeto de la clase Loan pasándole como parámetros al socio y al libro. Esto crea el vínculo de datos.
6. **Return message loan:** Se devuelve la confirmación de la creación del préstamo al bibliotecario.
7. **Call message "gives book":** El bibliotecario entrega la copia del libro al socio. La flecha hacia el círculo negro indica el fin de la interacción.

- **Camino de fallo [isAvailable == false]**

8. **Call message "notifies situation":** Si el libro no está disponible, el sistema omite la creación del préstamo y simplemente emite un mensaje hacia el exterior notificando que la operación no se puede realizar.

Pregunta 2 (40 %)

Pregunta 2.1. (40 %)



Powered By Visual Paradigm Community Edition

Pregunta 3 (40 %)

Pregunta 3.1. (40 %)

